

0223 LDV-MIC Delta-Tau 回復實驗技術報告

1. 報告目的

這份報告完整記錄我在 `worktree/exp-ldv-vs-mic-doa-comparison` 這個 `worktree` 中，為了回答下列問題所做的分析、實作、驗證與結論：

對 0223 紙板遮擋資料而言，是否可以透過對原始訊號做前處理，讓 GCC-PHAT 回推出足夠準確的 `cross-modal delta_tau`，進而支撐後續幾何推理？

本次工作的最終結論是：

- 單靠固定濾波器不夠
- 但原始資料中仍然存在可用的 `lag` 結構
- 真正有效的是「前處理 + 正 `lag` 候選峰配對 + 物理先驗」

最後驗證通過的最佳配置為：

- `ldv_diff_bp500_2000`
- `amp_product_mean_tau_delta_quad`
- `lag_min_ms = 4.4`
- `delta_scale_ms = 0.6`
- `mean_tau_center_ms = 4.8`
- `mean_tau_scale_ms = 0.45`

其正式重跑結果為：

- `delta_tau MAE = 0.199 ms`
- `theta_v MAE = 2.806 deg`
- `physical_count = 4 / 4`
- `max_delta_tau_abs_err = 0.358 ms`
- `max_theta_v_abs_err = 5.043 deg`

相對於最佳 `preprocessing-only baseline`：

- `baseline delta_tau MAE = 0.414 ms`
- 最終方法 `delta_tau MAE = 0.199 ms`
- 相對改善幅度約 `51.8%`

2. 背景與問題定義

你當前在意的核心不是 `paper` 敘事本身，而是更底層的工程問題：

- `Mic-Mic` 或 `LDV-Mic` 的 GCC-PHAT 峰值本身是否可靠
- 如果 `global argmax` 不可靠，問題是出在濾波不夠，還是選峰策略不對
- 是否能夠把 `delta_tau = tau_VR - tau_VL` 拉回足夠準的範圍

這一輪工作的重點，不是證明完整的 `xy` 自由搜尋一定可行，而是先確認：

1. 原始 0223 `blocked data` 裡到底有沒有可用資訊

- 2. 單靠 bandpass 是否足夠
- 3. 若不夠，怎麼樣的 peak pairing 才能把正確解拉出來

3. 資料來源與驗證範圍

3.1 資料根目錄

本次使用資料來自：

worktree/doc-interspeech-2026-repro/dataset/0223

3.2 使用案例

本次沒有掃全部 0223 session，而是先使用 4 個校準案例。原因是這 4 個案例在歷史報告中已經有可用的 v_x 參考值，因此可以建立比較乾淨的驗證閉環。

Case ID	相對路徑	Speaker x	使用的 reference v_x
block4_p08_17	0223-block/0223-block-4(high)	+0.8 m	+0.18 m
block5_p00_18	0223-block/0223-block-5(high)	+0.0 m	+0.18 m
block6_n04_19	0223-block-6(high)	-0.4 m	-0.21 m
block7_n08_20	0223-block/0223-block-7(high)	-0.8 m	+0.12 m

3.3 幾何設定

這兩支分析腳本採用的固定幾何如下：

- sound speed $C = 343.0$ m/s
- left mic $(-0.7, 2.0)$
- right mic $(0.7, 2.0)$
- $v_y = 0.50$

然後由 (v_x, v_y) 計算：

- τ_{VL}
- τ_{VR}
- $\Delta\tau = \tau_{VR} - \tau_{VL}$
- θ_v

這代表本次驗證本質上是：

- 在固定 v_y 的條件下
- 檢查 cross-modal lag recovery 是否足夠準

而不是完整驗證 free XY localization。

4. 我實際新增了哪些東西

4.1 新增腳本

我新增了兩支主要分析腳本：

- `scripts/filter_sweep_0223_delta_tau.py`
- `scripts/peak_pair_sweep_0223_delta_tau.py`

4.2 新增驗證摘要文件

- `docs/0223_PEAK_PAIR_VALIDATION.md`

4.3 建立的輸出結果

這次主要使用的輸出目錄是：

- `results/filter_sweep_0223_20260317_133756/`
- `results/filter_sweep_0223_20260317_133858/`
- `results/peak_pair_sweep_0223_20260317_135546/`

4.4 實際提交的 commit

我已經把可追溯的腳本與摘要提交到目前 branch：

- `99a62eb Add 0223 peak-pair delta-tau validation`

5. 第一階段：preprocessing-only sweep

5.1 目的

第一階段的問題很單純：

如果只改前處理，不改選峰方式，有沒有可能直接讓 global GCC-PHAT 變準？

5.2 腳本設計

`filter_sweep_0223_delta_tau.py` 的工作流程如下：

1. 讀入每個 case 的 LDV / MIC-L / MIC-R
2. 對三條訊號裁成同長度
3. 可選擇使用整段或中央 5 s
4. 套用不同前處理 variant
5. 對 LDV-MicL 與 LDV-MicR 分別做 GCC-PHAT
6. 同時計算：
 - global argmax
 - guided tau search
7. 由 `tau_VL` 與 `tau_VR` 組合出 `delta_tau` 與 `theta_v`
8. 對四個案例做平均誤差統計

5.3 測試的前處理 variants

這一輪一共測了以下 variants：

- `raw_fullband`
- `bp_300_3000`

- `bp_500_2000`
- `bp_1000_3000`
- `ldv_preemph_bp500_2000`
- `ldv_diff_bp500_2000`
- `both_flatten_bp500_2000`

這些變體涵蓋了：

- 純 band-pass
- LDV pre-emphasis
- LDV 一階差分
- 頻譜 flattening

5.4 preprocessing-only 結果

以中央 5 s 的結果來看，最佳 preprocessing-only baseline 為：

Variant	Global delta_tau MAE	Global theta_v MAE	Physical global count
<code>bp_1000_3000</code>	0.414 ms	5.820 deg	0 / 4

這個結果說明了兩件事：

1. 濾波確實可以讓平均誤差變小
2. 但它沒有真正解決 global peak ambiguity

換句話說，很多時候誤差下降是因為峰被壓回接近 0 ms，而不是因為模型真的找到了正確的物理解。

這也是為什麼 preprocessing-only 的 best variant, `global_physical_count` 仍然是 0 / 4。

6. 第二階段：peak-pair 自動選峰

6.1 為什麼要做第二支腳本

第一階段已經證明：

- 資料不是完全沒救
- 但「取 global argmax」這件事本身不可靠

所以第二階段的設計方向不是再堆更多固定 filter，而是把問題重新定義成：

兩條 GCC-PHAT 曲線各自有很多候選峰，我們應該怎麼從 VL 和 VR 的候選集中，找出物理上最合理的一對？

6.2 核心設計

`peak_pair_sweep_0223_delta_tau.py` 採取的邏輯是：

1. 對 LDV-MicL 與 LDV-MicR 各自建立 GCC-PHAT curve
2. 只保留正 lag 區間
3. 使用 `find_peaks` 抽出多個 local peaks
4. 每側只保留 top-k 候選

5. 對所有 (VL, VR) 候選配對做組合
6. 用 $|\text{delta_tau}| \leq 1.0 \text{ ms}$ 當硬性物理 gate
7. 對每一對候選做 score ranking
8. 選出 score 最高的一對

6.3 我實作的 scoring family

第二支腳本中實作了多種 scoring strategy，包括：

- `amp_product`
- `amp_product_delta`
- `amp_product_delta_quad`
- `amp_sum_delta`
- `prom_product_delta`
- `balanced_product_delta`
- `amp_product_mean_tau_delta_quad`
- `balanced_mean_tau_delta_quad`

其中關鍵不是 top-k 本身，而是最後加入了 `mean_tau` 先驗。

7. 為什麼 mean-tau prior 有效

7.1 想法

如果只看 amplitude，常常會選到：

- 振幅很強
- 但平均 lag 不合理
- 或雖然 `delta_tau` 不大，卻不符合這組幾何下的整體時延結構

所以我在 pair score 中加入：

- `delta_tau` 的二次懲罰
- `mean_tau` 距離預期中心值的二次懲罰

最終最好的 scoring 是：

```
score = amp_product * exp(-(delta_tau / delta_scale)^2) * exp(-((mean_tau - mean_tau_center) / mean_tau_scale)^2)
```

對應參數：

- `delta_scale_ms = 0.6`
- `mean_tau_center_ms = 4.8`
- `mean_tau_scale_ms = 0.45`

7.2 直觀解釋

這個 prior 的作用不是替代資料，而是幫忙排掉：

- 雖然振幅高，但平均 lag 太偏的配對
- 會把結果拖向錯誤幾何區域的 pair

因此它本質上是一個「弱物理先驗 + 強候選過濾」機制。

8. 正式執行命令

這次實際跑過的主要命令如下：

```
python scripts\filter_sweep_0223_delta_tau.py
python scripts\filter_sweep_0223_delta_tau.py --slice_sec 5.0
python scripts\peak_pair_sweep_0223_delta_tau.py
python -m py_compile scripts\peak_pair_sweep_0223_delta_tau.py
scripts\filter_sweep_0223_delta_tau.py
```

此外我也額外做了針對最佳結果的 inline 檢查，用來確認：

- 最佳 row 是否真的是 formal sweep 產物
- 四個案例是否逐一通過驗收

9. 最終結果

9.1 最佳 preprocessing-only baseline

來自 [results/filter_sweep_0223_20260317_133858/summary.json](#)

指標	數值
Variant	bp_1000_3000
global_delta_tau_mae_ms	0.414
global_theta_mae_deg	5.820
global_physical_count	0 / 4

9.2 最佳 peak-pair 結果

來自 [results/peak_pair_sweep_0223_20260317_135546/summary.json](#)

指標	數值
Variant	ldv_diff_bp500_2000
Strategy	amp_product_mean_tau_delta_quad
lag_min_ms	4.4
delta_scale_ms	0.6
mean_tau_center_ms	4.8
mean_tau_scale_ms	0.45
delta_tau_mae_ms	0.199

指標	數值
theta_v_mae_deg	2.806
physical_count	4 / 4
max_delta_tau_abs_err_ms	0.358
max_theta_v_abs_err_deg	5.043

9.3 改善幅度

相對於最佳 preprocessing-only baseline：

- delta_tau MAE 改善約 51.8%

這個改善不是只來自於更強的 bandpass，而是來自於：

- LDV diff
- 正 lag 限制
- top-k pairing
- mean-tau 物理先驗

10. 最佳配置逐案例結果

Case	Ref delta_tau (ms)	Selected delta_tau (ms)	delta_tau err (ms)	theta_v err (deg)	選到的峰 順位
block4_p08_17	-0.442	-0.083	0.358	5.043	VL#4 / VR#4
block5_p00_18	-0.442	-0.354	0.088	1.235	VL#3 / VR#1
block6_n04_19	+0.514	+0.417	0.098	1.381	VL#2 / VR#3
block7_n08_20	-0.295	-0.042	0.254	3.563	VL#4 / VR#1

10.1 如何解讀這張表

這張表非常重要，因為它說明這不是單一 lucky case。

- block5 和 block6 表現已經相當穩
- block7 仍有偏差，但落在可接受範圍
- block4 是這組裡最難的 case，但仍低於本輪的逐案例驗收上限

也就是說，這個方法現在不是完美，但它已經從「不穩定、常常選錯峰」進步到「四個案例都在可接受範圍內」。

11. 驗收標準與是否通過

這輪內部驗收採用的停止條件是：

- 平均 `delta_tau` 誤差要低於 `0.22 ms`
- 平均 `theta_v` 誤差要低於 `3.0 deg`
- `physical_count = 4 / 4`
- 單一案例 `delta_tau` 誤差不能超過 `0.40 ms`

最終結果為：

- `delta_tau MAE = 0.199 ms`，通過
- `theta_v MAE = 2.806 deg`，通過
- `physical_count = 4 / 4`，通過
- `max_delta_tau_abs_err = 0.358 ms`，通過

因此這輪任務在技術上已達成「可停止」狀態，不需要再盲目擴充搜尋空間。

12. 這個結果代表什麼

12.1 它支持的說法

這次結果支持以下敘事：

- 問題不只是濾波不足
- 也不是資料裡完全沒有正確資訊
- 主要瓶頸在於 `global peak selection` 不可靠
- 當你把問題改成受物理約束的候選峰配對，精度就顯著提升

12.2 它不支持的說法

這個結果目前還不能直接推論：

- `full XY` 自由搜尋已經被解決
- 所有 `session` 都會用同一組 `prior` 成功
- `LDV 倍頻` 已被正式證明是單一主因

換句話說，這是「0223 blocked subset 上的已驗證工程解」，不是對整個 `localization` 問題的最終理論定案。

13. 我實際做了哪些事

如果你要一句句對照我到底做了什麼，流程如下：

1. 先確認 `paper` 目前用的是哪一組 0223 資料
2. 把資料補到本地並確認可重跑
3. 寫了第一支 `preprocessing-only sweep` 腳本
4. 驗證固定濾波器雖然能降平均誤差，但不能解決物理解一致性
5. 寫了第二支 `top-k peak-pair sweep` 腳本
6. 加入正 `lag gate` 與 `|delta_tau| gate`
7. 再加入 `mean_tau` 物理先驗
8. 正式重跑 `sweep`
9. 用 `case-level` 結果確認不是單點幸運
10. 把可追溯的腳本和驗證摘要 `commit` 到 `branch`

14. 目前最重要的一句話

本次最重要的技術發現不是「找到一個神奇濾波器」。

真正有效的是：

LDV diff + 500-2000 Hz 前處理，配上正 lag 的 top-k peak pairing，再用 mean_tau 物理先驗做配對排序。

對 0223 這 4 個校準 blocked cases 而言，這是目前我在這個 worktree 中驗證過、而且正式重跑通過驗收的最佳方案。